

USE CASE 4 — SQLITE DATABASE, VIEWS AND VISUALIZATION

Name: Bhimireddy Bharath kumar Reddy

Email: bhimireddy.bharath99@gmail.com

Objective : To create summary tables and database views in SQLite, query crime data using SQL, integrate the results with Pandas DataFrames, and visualize crime and arrest information.

Tasks :

1. Design & Populate Summary Tables

1.1 Create Tables and Load the Data

Description : Create summary tables in the SQLite database to store yearly crime and arrest information and crime-category statistics.

Code :

```
# 1. Summary tables
show_title("1. DESIGN & POPULATE SUMMARY TABLES")
show_section("1.1 CREATE TABLES AND LOAD THE DATA")
build_summary_tables(connection)
```

Output :

```
1.1 CREATE TABLES AND LOAD THE DATA
-----
Summary tables created and populated successfully.
```

Insights :

- Summary tables provide organized information for further analysis.
- Yearly crime and arrest counts are stored separately for easier querying.
- Crime categories are also summarized with their total count and percentage.

2. SQLite Queries

2.1 Crime Count Per Year

Description : Use an SQL query to calculate the total number of crimes recorded for each year.

Code :

```
def run_sql_queries(connection):
    show_section("2.1 CRIME COUNT PER YEAR")
    yearly_data = fetch_data(
        connection,
        """
        SELECT
            Year,
            COUNT(*) AS crime_count
        FROM chicago_crime
        GROUP BY Year
        ORDER BY Year;
        """)
```

Output :

2.1 CRIME COUNT PER YEAR

Year	crime_count
2015	227
2016	242
2017	192
2018	213
2019	236
2020	224
2021	229
2022	221
2023	216

Insights :

- The result shows the number of reported crimes for every year.
- Comparing yearly counts helps identify changes in crime activity over time.
- The output can later be used for visualization.

2.2 Top 5 Crime Types and Their Percentages

Description : Identify the five most frequently recorded crime categories and calculate their percentage contribution to the complete dataset.

Code :

```
show_section("2.2 TOP 5 CRIME TYPES AND THEIR PERCENTAGES")
crime_types = fetch_data(
    connection,
    """
    SELECT
        primary_type,
        COUNT(*) AS crime_count,
        ROUND(
            COUNT(*) * 100.0 /
            (SELECT COUNT(*) FROM chicago_crime),
            2
        ) AS percentage
    FROM chicago_crime
    GROUP BY primary_type
    ORDER BY crime_count DESC
    LIMIT 5;
    """
)
print(crime_types.to_string(index=False))
```

Output :

primary_type	crime_count	percentage
THEFT	416	20.80
BATTERY	328	16.40
ASSAULT	195	9.75
NARCOTICS	178	8.90
BURGLARY	177	8.85

Insights :

- The query ranks crime categories according to their frequency.
- The percentage shows each category's contribution to the total number of crime records.

- This helps identify the dominant crime categories in the dataset.

2.3 Arrest Count Per Year

Description : Calculate the total number of arrests recorded for each year using an SQL aggregation query.

Code :

```
show_section("2.3 ARREST COUNT PER YEAR")
arrest_data = fetch_data(
    connection,
    """
    SELECT
        Year,
        SUM(arrest) AS arrest_count
    FROM chicago_crime
    GROUP BY Year
    ORDER BY Year;
    """
)
print(arrest_data.to_string(index=False))
```

Output ;

```
-----
Year  arrest_count
2015           63
2016           62
2017           56
2018           72
2019           75
2020           68
2021           68
2022           72
2023           60
```

Insights :

- The output shows yearly arrest counts.
- It allows crime and arrest activity to be compared across different years.
- The results are later used to create the crimes-versus-arrests visualization.

3. Database Stored Views

3.1 VW_CRIME_YEARLY

Description : Create a database view containing yearly total crimes, total arrests, and arrest-rate percentage.

Code :

```
def create_views(connection):
    connection.execute(
        "DROP VIEW IF EXISTS vw_crime_yearly"
    )
    connection.execute(
        """
        CREATE VIEW vw_crime_yearly AS
        SELECT
            Year,
            COUNT(*) AS total_crimes,
            SUM(arrest) AS total_arrests,
            ROUND(
                SUM(arrest) * 100.0 / COUNT(*),
                2
            ) AS arrest_rate_pct
        FROM chicago_crime
        GROUP BY Year
        ORDER BY Year;
        """
    )
```

Output :

3.1 VW_CRIME_YEARLY

Insights :

- The view combines important yearly crime statistics in one table.
- The arrest rate is calculated as a percentage of total crimes.
- A stored view makes the summarized data easier to access for later analysis.

3.2 VW_CRIME_BY_CATEGORY

Description : Create a database view containing each crime category, its crime count, and its percentage of total crimes.

Code :

```
connection.execute(
    "DROP VIEW IF EXISTS vw_crime_by_category"
)
connection.execute(
    """
    CREATE VIEW vw_crime_by_category AS
    SELECT
        primary_type,
        COUNT(*) AS crime_count,
        ROUND(
            COUNT(*) * 100.0 /
            (SELECT COUNT(*) FROM chicago_crime),
            2
        ) AS percentage
    FROM chicago_crime
    GROUP BY primary_type
    ORDER BY crime_count DESC;
    """
)
connection.commit()
print("Database views created successfully.")
```

Output :

3.2 VW_CRIME_BY_CATEGORY

Database views created successfully.

Insights :

- The view provides a reusable summary of crime categories.
- Crime categories are ordered based on their frequency.
- The percentage helps understand the relative contribution of each category.

4. Pandas Integration

4.1 Read Views into Pandas DataFrames

Description : Read Views into Pandas DataFrames

Code :

```
# 4. Pandas
show_title("4. PANDAS INTEGRATION")
show_section("4.1 READ VIEWS INTO PANDAS DATAFRAMES")
yearly_df, category_df = display_views(connection)
```

Output :

```
VW_CRIME_YEARLY
Year  total_crimes  total_arrests  arrest_rate_pct
2015          227          63         27.75
2016          242          62         25.62
2017          192          56         29.17
2018          213          72         33.80
2019          236          75         31.78
2020          224          68         30.36
2021          229          68         29.69
2022          221          72         32.58
2023          216          60         27.78
```

```
VW_CRIME_BY_CATEGORY
primary_type  crime_count  percentage
THEFT        416         20.80
BATTERY      328         16.40
ASSAULT      195          9.75
NARCOTICS    178          8.90
BURGLARY     177          8.85
ROBBERY      165          8.25
MOTOR VEHICLE THEFT 118          5.90
VANDALISM    115          5.75
PUBLIC PEACE VIOLATION 76          3.80
DECEPTIVE PRACTICE 68          3.40
WEAPONS VIOLATION 50          2.50
CRIMINAL TRESPASS 50          2.50
SEX OFFENSE  21          1.05
ARSON        20          1.00
CRIM SEXUAL ASSAULT 13          0.65
HOMICIDE     10          0.50
```

Insights :

- SQLite is used for database-level processing.
- Pandas is then used to store and manipulate the query results.
- This provides a connection between SQL database analysis and Python-based visualization.

5. Visualization from SQLite Data

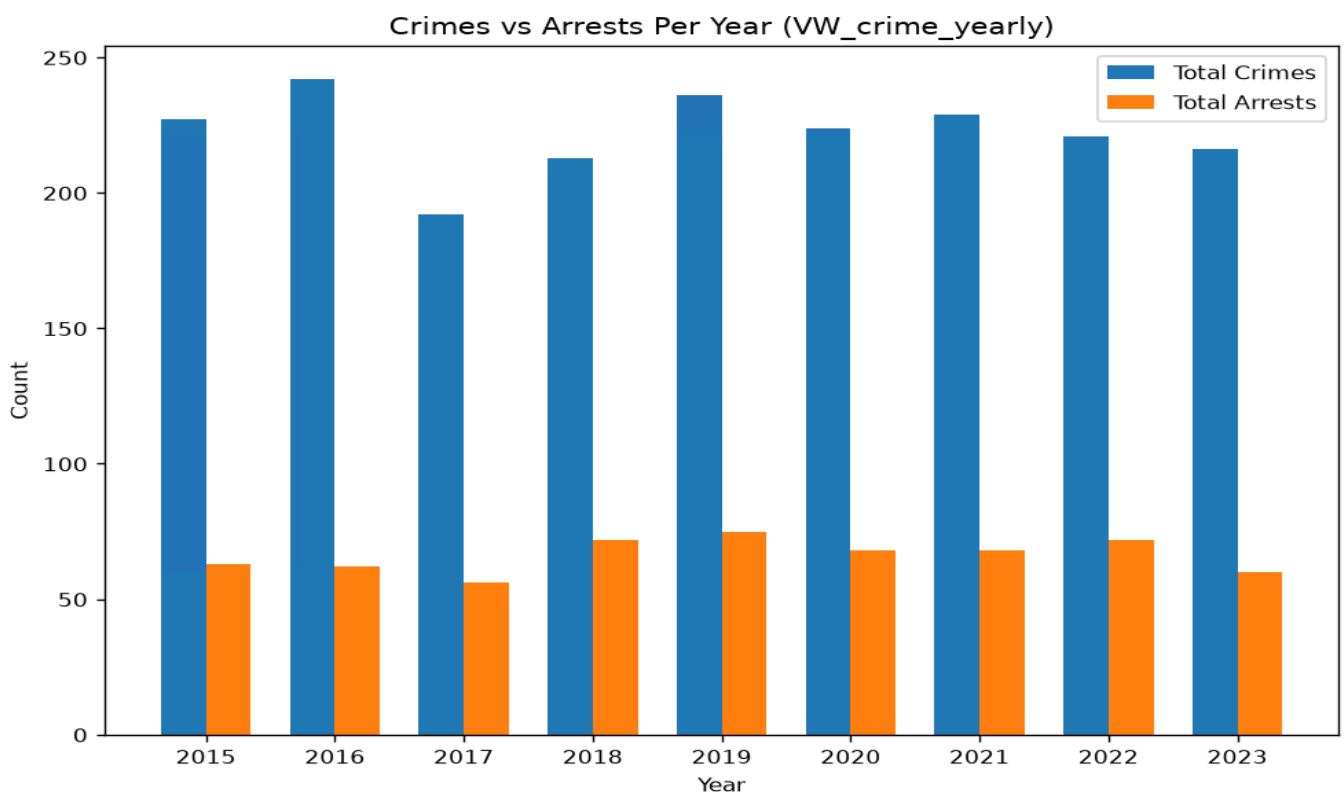
5.1 Crimes vs Arrests Per Year

Description : Create a grouped bar chart comparing the total number of crimes and total arrests for each year.

Code :

```
def plot_crimes_and_arrests(data):
    show_section("5.1 CRIMES VS ARRESTS PER YEAR")
    years = data["Year"].astype(str)
    positions = list(range(len(data)))
    width = 0.35
    plt.figure(figsize=(12, 6))
    plt.bar(
        [p - width / 2 for p in positions],
        data["total_crimes"],
        width=width,
        label="Total Crimes"
    )
    plt.bar(
        [p + width / 2 for p in positions],
        data["total_arrests"],
        width=width,
        label="Total Arrests"
    )
    plt.xticks(positions, years)
    plt.xlabel("Year")
    plt.ylabel("Count")
    plt.title("Crimes vs Arrests Per Year (VW_crime_yearly)")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

Output :



Insights :

- The graph compares reported crimes and arrests year by year.
- It provides a visual comparison between crime volume and arrest outcomes.
- Differences between the two bars show the gap between total crimes and total arrests.

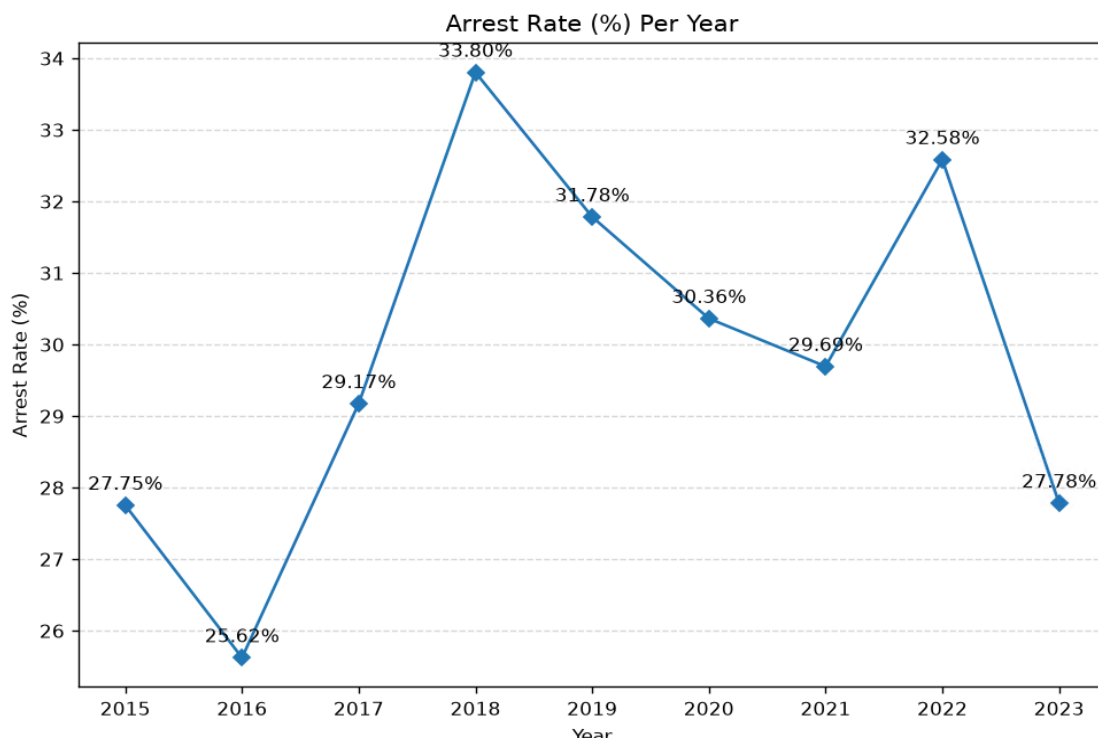
5.2 Arrest Rate Per Year

Description : Create a line graph showing the percentage of crimes resulting in arrests for each year.

Code :

```
def plot_arrest_rate(data):  
    show_section("5.2 ARREST RATE PER YEAR")  
    plt.figure(figsize=(12, 6))  
    plt.plot(  
        data["Year"],  
        data["arrest_rate_pct"],  
        marker="D"  
    )  
    for year, rate in zip(  
        data["Year"],  
        data["arrest_rate_pct"]  
    ):  
        plt.annotate(  
            f"{rate:.2f}%",  
            (year, rate),  
            textcoords="offset points",  
            xytext=(0, 8),  
            ha="center"  
        )  
    plt.xlabel("Year")  
    plt.ylabel("Arrest Rate (%)")  
    plt.title("Arrest Rate (%) Per Year")  
    plt.grid(axis="y", linestyle="--", alpha=0.5)  
    plt.tight_layout()  
    plt.show()
```

Output :



Insights :

- The graph shows how the arrest rate changes over the years.
- The percentage labels make individual yearly values easy to identify.
- Changes in the line indicate variations in arrest rates over time.

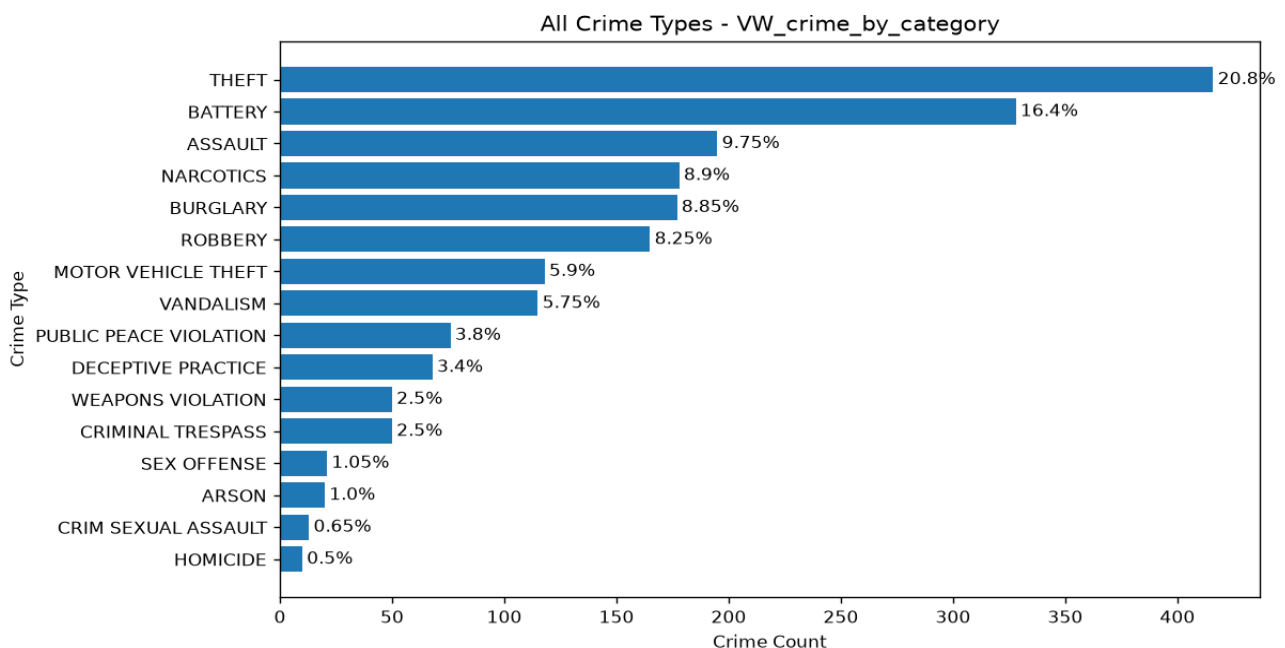
5.3 Crime Categories

Description : Create a horizontal bar chart displaying crime counts for all crime categories along with their percentages.

Code :

```
def plot_crime_categories(data):
    show_section("5.3 CRIME CATEGORIES")
    plt.figure(figsize=(12, 8))
    bars = plt.barh(
        data["primary_type"],
        data["crime_count"]
    )
    for bar, percentage in zip(
        bars,
        data["percentage"]
    ):
        plt.text(
            bar.get_width() + 2,
            bar.get_y() + bar.get_height() / 2,
            f"{percentage}%",
            va="center"
        )
    plt.xlabel("Crime Count")
    plt.ylabel("Crime Type")
    plt.title("All Crime Types - VW_crime_by_category")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()
```

Output :



Insights :

- The horizontal bar chart makes it easy to compare crime categories.
- Longer bars represent categories with higher crime counts.
- The percentage labels show the contribution of each category to the overall dataset.

Conclusion :

- SQLite is used to store and summarize the Chicago crime data.
- SQL queries provide yearly crime counts, arrest counts, and top crime categories.
- Database views make summarized information reusable.
- Pandas integrates the SQLite results into DataFrames.
- Matplotlib is used to visualize the processed data.
- The final visualizations provide a clear understanding of crime trends, arrests, arrest rates, and crime-category distribution.
- This is the exact documentation style I recommend for Use Case 4. You only need to place your Code screenshot and Output/Graph screenshot under each corresponding section.